

머신러닝 모델 설명

08/18/2026 16:56:49

<https://chatgpt.com/c/6a7e7068-e58c-83ee-8fb5-a4bc0f3254f7>

좋아. DAY2도 DAY1이랑 같은 방식으로 볼게.이번에도 먼저 전체 챕터 흐름부터 잡고,...



좋아. DAY2도 **DAY1**이랑 같은 방식으로 볼게.

이번에도 먼저 전체 챕터 흐름부터 잡고, 각 챕터마다 왜 하는지 → 뭔지 → 코드 한 줄씩 → 결과 해석 → **DAY1과 연결** 순서로 설명할게.

DAY2는 DAY1보다 한 단계 올라갔어. 핵심은 “모델을 만드는 것”에서 끝나지 않고, 그 모델을 제대로 평가하고 → 성능을 조절하고 → **Decision Tree**의 설정값까지 튜닝하는 것이야.

특히 이번 파일에서는 정확도만 보면 왜 위험한가, 정밀도·재현율·F1·ROC-AUC가 왜 필요한가, GridSearchCV가 뭘 찾는지 가 핵심이야.

Thinking



ZIP 파일 내용 목록 확인

- 노트북과 ZIP 파일 내용을 추출하고 셀 출력 점검 완료



Inspecting and Explaining Diabetes Analysis Notebooks

- 구성 정리하기

코드와 개념을 세 장으로 나누고, 각 부분을 순서대로 설명하려고 해요. 정확도와 혼동행렬부터 당뇨 예측, 결정 트리와 GridSearchCV까지 다루며 ROC 축 표기의 오류도 짚겠습니다.

- 구성을 정리했습니다
- 당뇨병 분석 노트북을 점검하고 설명함

✓ 54s 동안 처리함

좋아. DAY2는 DAY1보다 내용이 많지만, 큰 그림부터 잡으면 생각보다 연결이 잘 돼.

DAY1에서 배운 게:

데이터 → X/y → train/test → 모델 → fit → predict → accuracy

였다면 DAY2는 여기서 질문이 하나 더 생긴 거야.

“정확도가 높으면 정말 좋은 모델일까?”

그리고 뒤로 갈수록:



“Decision Tree를 어떻게 조절하면 더 좋은 모델을 만들 수 있을까?”

까지 간다.

DAY2 파일은 크게 **3챕터**야.

DAY2 전체 지도

Chapter 1. 정확도 / Precision / Recall / F1 / ROC-AUC

왜?

모델을 정확도 하나로만 평가하면 잘못 판단할 수 있기 때문.

배우는 것:

- Accuracy
- Confusion Matrix
- Precision
- Recall
- Threshold
- F1 Score
- ROC Curve
- AUC

Chapter 2. 피마 인디언 당뇨병 예측

왜?

Chapter 1에서 배운 평가 방법을 실제 분류 문제에 적용해보기 위해서.

- 당뇨병 데이터
 - 전처리
 - Logistic Regression
 - 예측
 - Precision / Recall / F1 / AUC
 - Threshold 조절

Chapter 3. Decision Tree

왜?

DAY1에서 그냥 사용했던 Decision Tree가

어떻게 판단하는지

어떤 Feature를 중요하게 보는지

설정값을 어떻게 조정하는지

배우는 챕터야.

마지막에는:

```
GridSearchCV
```

까지 나온다.

이건 좋은 모델 설정값을 자동으로 찾아주는 기능이다.

Chapter 1

01_정확도_ROC_AUC.ipynb

1. 정확도만 보면 왜 안 되는가?

첫 번째 코드부터 목적이 특이해.

```
import sklearn
import numpy as np
from sklearn.base import BaseEstimator
```

numpy 와 sklearn을 가져온다.

여기서:

```
BaseEstimator
```

는 sklearn 모델처럼 행동하는 사용자 정의 모델을 만들기 위해 가져온 것이다.

1-1. 일부러 허접한 모델 만들기

```
class MyDummyClassifier(BaseEstimator):
```

직접 MyDummyClassifier 라는 모델 클래스를 만든다.

Dummy 라는 이름에서 알 수 있듯 제대로 학습하는 모델이 아니다.

```
def fit(self, X, y=None):  
    pass
```

원래:

```
model.fit(X_train, y_train)
```

하면 학습해야 하지?

그런데 여기서는:

```
pass
```

아무것도 안 한다.

즉,

이 모델은 실제로 데이터를 공부하지 않는다.

그런데 예측은 한다.

```
def predict(self, X):
```

새로운 데이터를 받는다.

```
pred = np.zeros((X.shape[0], 1))
```

예측 결과를 저장할 공간을 만든다.

예를 들어 X에 승객이 179명이면:

```
179행 × 1열
```

짜리 공간을 만든다.

처음에는 전부 0.

그리고:

```
for i in range(X.shape[0]):
```

승객을 한 명씩 본다.

```
if X["Sex"].iloc[i] == 1:
    pred[i] = 0
else:
    pred[i] = 1
```

이 모델의 규칙은 그냥 이것뿐이야.

남자 → 사망(0)
여자 → 생존(1)

다른 데이터는 하나도 안 본다.

그런데 Titanic에 적용하면:

```
print(accuracy_score(y_test, mypred))
```

결과:

0.7821

즉,

78.2% 정확도

가 나온다.

여기서 교수님이 보여주려는 것

이 모델은 머신러닝 학습을 사실상 하나도 안 했다.

그런데 정확도가 78%다.

그러면:

정확도 78%니까 좋은 AI 모델이네?

라고 말하기는 이상하지.

따라서:

Accuracy 하나만 보고 모델을 평가하면 안 된다.

를 보여주는 첫 번째 예제야.

1-2. 더 극단적인 예제

이번에는 숫자 이미지 데이터가 나온다.

```
from sklearn.datasets import load_digits
```

0~9 숫자 손글씨 데이터다.

```
digits = load_digits()
```

데이터를 불러온다.

```
print(digits.data.shape)
```

결과:

```
(1797, 64)
```

즉:

```
숫자 이미지 1797개  
Feature 64개
```

가 있다.

8×8 이미지니까:

```
8 × 8 = 64
```

픽셀값이다.

정답은:

```
digits.target
```

예를 들어:

```
0 1 2 3 ... 8 9
```

그런데 이번에는 문제를 바꾼다.

```
y = (digits.target == 7).astype(int)
```

이 코드가 중요해.

먼저

```
digits.target == 7
```

하면:

```
7이면 True  
7이 아니면 False
```

가 된다.

그 다음:

```
.astype(int)
```

하면:

```
True → 1  
False → 0
```

이 된다.

즉 문제를:

“이 숫자가 7인가 아닌가?”

로 바꾼 것이다.

데이터를 나눈다.

```
X_train, X_test, y_train, y_test = train_test_split(
    digits.data,
    y,
    random_state=42
)
```

DAY1과 똑같지.

시험 데이터 정답을 보면:

```
pd.Series(y_test).value_counts()
```

결과:

0	409
1	41

즉 450개 중:

7 아님	= 409개
7	= 41개

야.

가짜 모델

```
class MyFakeClassifier(BaseEstimator):
```

역시 가짜 모델.

```
def predict(self, X):  
    return np.zeros((len(X), 1), dtype=bool)
```

무조건:

```
0  
0  
0  
0  
0  
...
```

이라고 예측한다.

즉:

“무조건 7이 아니다.”

라고 답하는 모델이다.

그런데 정확도가:

0.9089

즉:

90.9%

가 나온다.

왜?

실제로 450개 중 409개가 7이 아니었으니까.

$409 / 450 \approx 90.9\%$

중요한 결론

이 모델은 7을 단 한 번도 맞히지 못한다.

그런데 정확도는 91%.

따라서:

*데이터의 클래스 비율이 불균형하면 **Accuracy**가 특히 위험하다.*

2. Confusion Matrix

그래서 등장하는 것이:

```
from sklearn.metrics import confusion_matrix
```

```
confusion_matrix(y_test, fakepred)
```

결과:

```
[[409  0]
 [ 41  0]]
```

처음 보면 무슨 암호 같지만 이렇게 읽어.

		모델 예측	
		0	1
실제 0	TN		FP
실제 1	FN		TP

따라서:

```
[[TN, FP],
 [FN, TP]]
```

이다.

이번 결과는:

```
[[409, 0],
```

[41, 0]]

즉:

TN = 409

FP = 0

FN = 41

TP = 0

TP / TN / FP / FN

이건 꼭 알아두자.

TP - True Positive

실제로 1이고 모델도 1.

실제 당뇨병 0

모델 당뇨병 0

정답.

TN - True Negative

실제 0이고 모델도 0.

실제 당뇨병 X

모델 당뇨병 X

정답.

FP - False Positive

실제로 0인데 모델은 1.

실제 정상

모델: 당뇨병입니다

오답.

FN - False Negative

실제로 1인데 모델은 0.

실제 당뇨병
모델: 정상입니다

오답.

특히 의료에서는 FN이 위험할 수 있겠지.

3. Precision - 정밀도

```
precision_score(y_test, fakepred)
```

정밀도는:

모델이 1이라고 한 것 중 실제로 1인 비율

이다.

공식은:

$$\frac{TP}{TP + FP}$$

예를 들어 모델이:

10명에게 당뇨병이라고 판단

했는데 실제 당뇨병이 8명이면:

$$\text{Precision} = 8 / 10 = 80\%$$

4. Recall - 재현율

```
recall_score(y_test, fakepred)
```

재현율은:

실제로 1인 사람 중 모델이 얼마나 찾아냈느냐

이다.

공식:

$$\frac{TP}{TP + FN}$$

예를 들어 실제 당뇨병 환자가 100명인데 모델이 90명을 찾아냈다면:

$$\text{Recall} = 90\%$$

정밀도와 재현율 차이

이렇게 생각하면 쉬워.

Precision

“내가 잡았다고 한 사람 중 진짜가 얼마나 많지?”

Recall

“진짜 환자들을 얼마나 놓치지 않고 잡았지?”

5. 평가 함수 만들기

```
def get_clf_eval(y_test, pred):
```

평가를 반복하기 귀찮으니까 함수로 만든다.

```
confusion = confusion_matrix(y_test, pred)
```

오차행렬.

```
accuracy = accuracy_score(y_test, pred)
```

정확도.

```
precision = precision_score(y_test, pred)
```

정밀도.

```
recall = recall_score(y_test, pred)
```

재현율.

즉 한 번에:

```
오차행렬  
정확도  
정밀도  
재현율
```

을 출력하려는 함수다.

6. Logistic Regression 다시 등장

Titanic 데이터를 사용한다.

```
lr_clf = LogisticRegression(solver="liblinear")
```

Logistic Regression 분류 모델을 만든다.

```
lr_clf.fit(X_train, y_train)
```

학습.

```
pred = lr_clf.predict(X_test)
```

예측.

결과:

```
[[90 15]
 [22 52]]
```

즉:

```
TN = 90
FP = 15
FN = 22
TP = 52
```

그리고:

```
Accuracy  = 0.793
Precision = 0.776
Recall    = 0.703
```

7. `predict()` 와 `predict_proba()` 차이

DAY2에서 매우 중요해.

DAY1에서는:

```
lr_clf.predict(X_test)
```

만 봤지.

결과:

```
0
1
0
1
...
```

즉 최종 답을 준다.

이번에는:

```
pred_prob = lr_clf.predict_proba(X_test)
```

확률을 가져온다.

예:

```
[0.883, 0.117]
```

뜻은:

0일 확률 = 88.3%

1일 확률 = 11.7%

다음 사람:

```
[0.30, 0.70]
```

이면:

0 확률 = 30%

1 확률 = 70%

그래서 보통:

1의 확률 $\geq 0.5 \rightarrow 1$

1의 확률 $< 0.5 \rightarrow 0$

으로 결정한다.

그 **0.5**가 임계값 **Threshold**야.

8. Threshold

```
custom_threshold = 0.5
```

판단 기준을 50%로 한다.

```
pred_prob_1 = pred_prob[:, 1].reshape(-1, 1)
```

여기서:

```
pred_prob[:, 1]
```

은 모든 사람에 대해:

1일 확률만 가져온다.

예:

```
0.116  
0.206  
0.148  
...
```

```
Binarizer(threshold=custom_threshold)
```

확률을 0/1로 바꿔준다.

예:

```
0.3 → 0  
0.7 → 1
```

Threshold를 0.4로 낮추면?

```
custom_threshold = 0.4
```

이제:

```
0.4 이상이면 1
```

이다.

그러면 이전에는:

0.45 → 0

였는데 이제는:

0.45 → 1

이 된다.

즉 **1**이라고 판단하는 사람이 많아진다.

결과:

Precision ↓

Recall ↑

실제 결과도:

Threshold 0.5:

Precision 0.776

Recall 0.703

Threshold 0.4:

Precision 0.740

Recall 0.770

재현율이 올라갔지.

이게 왜 중요할까?

예를 들어 암 검진이라면:

환자를 놓치는 것

이 매우 위험하다고 판단할 수 있다.

그러면 Threshold를 낮춰서:

의심되는 사람을 더 많이 잡는다.

그러면 Recall은 올라간다.

대신 정상인도 환자로 판단할 가능성이 늘기 때문에 Precision은 떨어질 수 있다.

9. Precision-Recall Curve

```
precision_recall_curve(...)
```

임계값을 계속 바꾸면서:

Precision은 어떻게 변하고
Recall은 어떻게 변하는지

계산한다.

그래프에서는 보통:

Threshold ↑
Precision ↑
Recall ↓

하는 경향을 볼 수 있다.

즉 둘 사이에는 **Trade-off**가 존재한다.

10. F1 Score

```
from sklearn.metrics import f1_score
```

F1은:

Precision과 Recall을 함께 고려한 점수

다.

공식은 시험에서 볼 수 있으니 알아두면 좋다.

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

지금은 이렇게 이해해.

Precision만 높은 모델, Recall만 높은 모델이 아니라 둘의 균형을 보자.

파일 결과:

```
F1 ≈ 0.738
```

11. ROC Curve

이번에는:

```
from sklearn.metrics import roc_curve
```

ROC 곡선이다.

ROC에서는:

```
TPR  
FPR
```

을 본다.

TPR

True Positive Rate.

사실상:

```
Recall
```

과 같다.

FPR

False Positive Rate.

실제로 0인데 1이라고 잘못 판단한 비율.

FP

FP + TN

좋은 모델은:

TPR은 높고

FPR은 낮은

모델이다.

그래서 ROC 그래프가 왼쪽 위에 가까울수록 좋다.

참고

노트북 그래프에:

```
plt.xlabel('FPRS(1 - Sensitivity)')
```

라고 되어 있는데 정확히는:

$FPR = 1 - Specificity$ (특이도)

야.

1 - Sensitivity 는 아니다.

코드 설명 문구의 작은 오류라고 보면 된다.

12. ROC-AUC

```
roc_auc_score(y_test, pred_proba)
```

AUC는:

ROC 곡선 아래 면적

이다.

대략:

0.5 → 랜덤 수준

1.0 → 완벽

Titanic Logistic Regression 결과:

AUC = 0.877

상당히 괜찮은 분리 능력을 보였다는 뜻이다.

Chapter 1 핵심

지금 시험 문제처럼 정리하면:

지표	질문
Accuracy	전체 중 얼마나 맞혔나
Precision	1이라고 한 것 중 진짜 1은?
Recall	실제 1 중 얼마나 찾아냈나
F1	Precision과 Recall 균형
ROC-AUC	모델의 전반적인 분류 능력

Chapter 2

02_피마인디언당뇨병예측.ipynb

이 챕터는 새로운 이론이라기보다:

Chapter 1을 실제 데이터에 적용

하는 실습이야.

1. 데이터 불러오기

```
diabetes_data = pd.read_csv('./data/diabetes.csv')
```

당뇨병 데이터를 읽는다.

정답 컬럼:

```
Outcome
```

이다.

```
0 = 당뇨병 아님  
1 = 당뇨병
```

```
diabetes_data['Outcome'].value_counts()
```

결과:

```
0 = 500  
1 = 268
```

즉 정상이 더 많다.

완전히 균등한 데이터는 아니다.

2. `info()`

```
diabetes_data.info()
```

확인하는 것:

```
행 수  
컬럼  
결측치  
데이터 타입
```

EDA할 때 하던 거랑 똑같아.

3. X와 y

```
X = diabetes_data.iloc[:, :-1]
```

`:-1`

→ 마지막 컬럼 전까지.

즉:

```
Pregnancies  
Glucose  
BloodPressure  
...  
Age
```

를 X로 사용한다.

```
y = diabetes_data.iloc[:, -1]
```

마지막 컬럼:

```
Outcome
```

을 y로 사용한다.

즉:

```
x = 환자 정보  
y = 당뇨병 여부
```

4. `stratify=y`

```
train_test_split(  
    X,  
    y,  
    test_size=0.2,  
    random_state=42,
```

```
stratify=y  
)
```

DAY1에서는 없었던 게:

```
stratify=y
```

야.

왜 넣었을까?

원래 데이터가:

```
0 약 65%  
1 약 35%
```

라면 train/test도 비슷한 비율로 나눠준다.

즉:

정답 클래스의 비율을 유지하면서 분리

한다.

분류에서 상당히 자주 사용한다.

5. Logistic Regression

```
lr_clf = LogisticRegression(solver='liblinear')
```

모델 생성.

```
lr_clf.fit(X_train, y_train)
```

학습.

```
pred = lr_clf.predict(X_test)
```

0/1 최종 예측.

```
pred_proba = lr_clf.predict_proba(X_test)[: , 1]
```

1, 즉 당뇨병일 확률.

결과:

```
Accuracy  0.708
Precision  0.588
Recall     0.556
F1         0.571
AUC        0.830
```

여기서 중요한 건:

정확도 70.8% 하나만 볼 게 아니라 다른 지표도 같이 본다.

6. describe()

```
diabetes_data.describe()
```

각 컬럼의:

```
평균
표준편차
최솟값
25%
중앙값
75%
최댓값
```

을 본다.

이게 바로 **EDA**다.

7. Glucose 분포

```
plt.hist(diabetes_data['Glucose'], bins=100)
```

혈당값이 어떻게 분포하는지 본다.

`bins=100` 은 히스토그램 막대를 얼마나 잘게 나눌지 지정한다.

8. 0값 확인

```
zero_features = [  
    'Glucose',  
    'BloodPressure',  
    'SkinThickness',  
    'Insulin',  
    'BMI'  
]
```

이 컬럼들은 원래 Pima 데이터에서는:

0이 실제 값으로 말이 안 되는 경우

가 존재할 수 있어서 확인하는 코드야.

예를 들어:

```
Glucose = 0  
BloodPressure = 0  
BMI = 0
```

는 현실적으로 이상하지.

그래서 원래 데이터셋에서는 **0을 결측치처럼** 처리하는 실습을 많이 한다.

그런데 중요한 점이 있어.

이번에 올린 `diabetes.csv` 를 직접 확인해보면:

```
Glucose      0개  
BloodPressure 0개  
SkinThickness 0개  
Insulin      0개  
BMI          0개
```

이미 0값이 없어.

즉 이 파일은 **사전에 어느 정도 처리된 버전**으로 보인다.

그래서:

```
replace(0, mean_zero_features)
```

를 실행해도 현재 파일에서는 사실상 바뀌는 값이 없다.

코드를 이해하는 목적은:

원본에서 비정상적인 0값이 있다면 평균 등으로 대체한다.

정도로 보면 돼.

9. StandardScaler

여기서 새로 등장한다.

```
from sklearn.preprocessing import StandardScaler
```

왜 하는가?

각 Feature의 숫자 크기가 너무 다르기 때문이다.

예:

```
Pregnancies = 5  
Glucose = 150  
Insulin = 500  
BMI = 30  
DiabetesPedigree = 0.5
```

크기 차이가 상당하지.

그래서 일정한 기준으로 맞춰준다.

이것을:

Scaling(스케일링)

이라고 한다.

```
scaler = StandardScaler()
```

스케일러 생성.

```
X_scaled = scaler.fit_transform(X)
```

두 가지를 한 번에 한다.

`fit`

→ 평균과 표준편차 파악

`transform`

→ 그 기준으로 값 변환

StandardScaler는 대략:

평균을 0 근처

표준편차를 1 근처

가 되도록 바꾼다.

지금은 공식보다:

Feature들의 숫자 규모를 비슷하게 맞춘다.

정도로 이해하면 충분해.

10. Threshold 실험

여기서 Chapter 1을 그대로 이용한다.

```
thresholds = [  
    0.3,  
    0.33,  
    0.36,  
    0.39,  
    0.42,  
    0.45,
```

```
0.48,  
0.50  
]
```

Threshold를 여러 개 시험한다.

0.5일 때:

```
Recall = 0.556
```

그런데 0.3이면:

```
Recall = 0.889
```

엄청 올라간다.

대신 Precision:

```
0.588 → 0.623
```

이 데이터에서는 같이 조금 올라갔지만 일반적으로 임계값을 낮추면 Recall은 올라가고 Precision은 내려가는 경향이 있다.

당뇨병 **검진용 모델**이라면:

환자를 놓치지 않는 게 중요하다.

고 판단할 경우 Recall을 상대적으로 중요하게 볼 수도 있다.

다만 실제 의료 시스템에서는 단순히 코드로 임계값 하나 정해서 사용할 문제가 아니고, 임상 목적·비용·후속검사 등을 함께 고려해야 한다.

Chapter 3

03_결정트리.ipynb

여기서부터 DAY1에서 사용했던 Decision Tree를 제대로 파고든다.

1. Decision Tree 다시 학습

```
dt_clf = DecisionTreeClassifier(random_state=42)
```

결정트리 생성.

```
iris_data = load_iris()
```

Iris 데이터.

```
train_test_split(...)
```

학습/테스트 나누기.

```
dt_clf.fit(X_train, y_train)
```

결정트리 학습.

여기까지 DAY1 복습이다.

2. Tree를 그림으로 보기

```
from sklearn.tree import export_graphviz
```

Decision Tree를 그림으로 표현할 데이터를 만든다.

```
export_graphviz(  
    dt_clf,  
    out_file="tree.dot",  
    ...  
)
```

학습된 Tree를:

```
tree.dot
```

파일로 저장한다.

그 다음:

```
graphviz.Source(dot_graph)
```

Tree 구조를 그림으로 보여준다.

보통 이런 식이야.

```
petal length <= 2.45?  
  /      \  
Yes      No  
  ↓      ↓  
setosa   또 다른 질문
```

즉 Decision Tree가 실제로 무슨 질문을 했는지 볼 수 있다.

3. Feature Importance

```
dt_clf.feature_importances_
```

새로운 개념이다.

모델이 예측할 때 각 Feature를 얼마나 중요하게 사용했는가

를 나타낸다.

결과:

```
sepal length = 0.000  
sepal width  = 0.017  
petal length = 0.906  
petal width  = 0.077
```

즉 이 Decision Tree는 거의:

petal length, 꽃잎 길이

를 중심으로 꽃을 구분했다.

약 90.6% 중요도다.

4. 가짜 분류 데이터 만들기

```
from sklearn.datasets import make_classification
```

실습하기 좋은 가상 데이터를 자동 생성한다.

```
X_features, y_labels = make_classification(  
    n_features=2,  
    ...  
    n_classes=3  
)
```

뜻:

Feature 2개
Class 3개

인 분류 데이터를 만들어라.

그래프에 찍으면 점들이 세 종류로 나뉜다.

5. Decision Boundary

```
visualize_boundary(...)
```

코드가 복잡하지만 함수 내부 전체를 지금 외울 필요 없다.

목적 하나만 알아두면 돼.

Decision Tree가 공간을 어떻게 잘라서 분류하는지 시각화

한다.

Decision Tree는 보통 이런 식으로 영역을 나눈다.

```
|-----|  
| class 0 | class1  
|         |
```

```
|_____|  
| class 2  
|_____|
```

직선 형태로 계속 공간을 자른다.

6. min_samples_leaf

처음:

```
DecisionTreeClassifier(random_state=42)
```

과

```
DecisionTreeClassifier(  
    min_samples_leaf=6,  
    random_state=42  
)
```

를 비교한다.

min_samples_leaf=6 뜻:

Tree의 마지막 잎(Leaf)에 최소 6개의 데이터가 있어야 한다.

왜 이런 제한을 걸까?

Decision Tree를 아무 제한 없이 키우면:

데이터 하나하나까지 완벽하게 외움

이 될 수 있다.

그게 바로:

Overfitting(과적합)

이다.

과적합

훈련 문제는 엄청 잘 풀지만 처음 보는 문제를 못 푸는 상태.

학생으로 비유하면:

```
기출 문제 답을 통째로 암기
→ 똑같은 문제 100점
→ 조금 바꾸면 못 풀
```

이다.

그래서:

```
min_samples_leaf
max_depth
min_samples_split
```

같은 설정으로 Tree가 지나치게 복잡해지지 않게 조절한다.

7. Human Activity 데이터

이제 실전 데이터가 나온다.

센서 데이터를 이용해서 사람의 행동을 맞힌다.

정답은:

```
ACTION_MAP = {
    "WALKING": 1,
    "WALKING_UPSTAIRS": 2,
    "WALKING_DOWNSTAIRS": 3,
    "SITTING": 4,
    "STANDING": 5,
    "LAYING": 6
}
```

즉:

센서 값을 보고 사람이 걷는 중인지, 앉아 있는지, 누워 있는지를 예측

하는 모델이다.

이건 **6개 Class**를 가진 다중분류다.

8. Feature가 561개

```
print(len(feature_name))
```

결과:

```
561
```

즉 모델에게 들어가는 정보가 무려 561개다.

예:

```
tBodyAcc-mean()-X  
tBodyAcc-mean()-Y  
tBodyAcc-std()-X  
...
```

스마트폰/센서 가속도 등의 여러 통계 특징이다.

9. 중복 컬럼 처리

```
feature_dup_df = feature_name_df.groupby('column_name').count()
```

같은 이름의 컬럼이 있는지 확인한다.

42개의 중복 이름이 존재한다.

DataFrame에서 동일한 컬럼명이 있으면 다루기 불편하니까:

```
이름  
이름_1  
이름_2
```

처럼 변경한다.

그 역할을 하는 함수가:

```
get_new_feature_name_df()
```

이다.

여기는 지금 코드 한 줄 한 줄 외울 필요가 낮아.

목적만:

중복 *Feature* 이름을 서로 다르게 만들어준다.

라고 이해하면 돼.

10. 데이터 구성 함수

```
get_human_dataset()
```

이 함수의 목적은 크게 네 가지다.

```
CSV 읽기
↓
Feature 이름 정리
↓
x와 y 분리
↓
문자 행동을 숫자로 변환
```

결과:

```
X_train = (7352, 561)
X_test  = (2947, 561)

y_train = (7352, 1)
y_test  = (2947, 1)
```

뜻:

```
학습 데이터 7352개
Feature 561개

테스트 데이터 2947개
Feature 561개
```

11. 첫 Decision Tree

```
dt_clf = DecisionTreeClassifier(random_state=42)
```

모델.

```
dt_clf.fit(X_train, y_train.astype(int))
```

학습.

`astype(int)` 는 행동 번호를 정수로 변환한다.

```
pred = dt_clf.predict(X_test)
```

예측.

```
accuracy_score(...)
```

결과:

```
0.8622
```

즉:

86.22%

12. Hyperparameter

이제 DAY2 후반부의 중요한 개념이 나온다.

```
dt_clf.get_params()
```

하면:

```
max_depth  
min_samples_leaf  
min_samples_split
```

```
criterion
...
```

같은 것들이 나온다.

이런 것을:

Hyperparameter(하이퍼파라미터)

라고 한다.

모델 파라미터와 Hyperparameter 차이

간단하게만 구분하자.

모델이 학습하면서 알아내는 것

```
Parameter
```

사람이 학습 전에 정해주는 설정

```
Hyperparameter
```

예를 들어 Decision Tree:

```
max_depth=8
```

은 우리가 지정하니까 Hyperparameter다.

13. `max_depth`

```
params = {
    'max_depth': [6, 8, 10, 12, 16, 20, 24]
}
```

Tree의 최대 깊이를 여러 개 시험하겠다는 뜻이다.

예:

```
depth 6
depth 8
depth 10
...
```

14. GridSearchCV

DAY2 핵심 중 하나.

```
from sklearn.model_selection import GridSearchCV
```

이게 뭐가?

*여러 **Hyperparameter** 조합을 자동으로 시험해서 가장 좋은 것을 찾아준다.*

예를 들어 우리가 직접:

```
depth=6 학습 → 평가
depth=8 학습 → 평가
depth=10 학습 → 평가
...
```

하기 귀찮잖아.

GridSearchCV가 대신 해준다.

```
grid_cv = GridSearchCV(
    dt_clf,
    param_grid=params,
    scoring='accuracy',
    cv=5,
    verbose=1
)
```

하나씩 보면:

```
dt_clf
```

조절할 모델:

Decision Tree

param_grid=params

시험할 설정:

max_depth = 6, 8, 10 ...

scoring='accuracy'

무슨 점수로 좋은 모델을 결정할까?

Accuracy

cv=5

DAY1에서 배운:

5-Fold Cross Validation

을 사용한다.

따라서:

max_depth 7가지
×
5번 교차검증
=
35번 학습

그래서 출력에:

35 fits

라고 나온다.

15. 가장 좋은 설정

```
grid_cv.best_score_
```

가장 좋은 교차검증 점수.

```
0.8479
```

```
grid_cv.best_params_
```

결과:

```
{'max_depth': 8}
```

즉 이 실험에서는:

Tree 깊이를 8로 했을 때 가장 좋았다.

16. 두 개의 Hyperparameter 동시에 찾기

이번에는:

```
params = {  
    'max_depth': [8, 12, 16, 20],  
    'min_samples_split': [16, 24]  
}
```

조합 수:

```
4 × 2 = 8
```

그리고 5-Fold니까:

```
8 × 5 = 40
```

그래서:

40 fits

결과:

```
{
    'max_depth': 8,
    'min_samples_split': 24
}
```

가 가장 좋았다.

`min_samples_split`

노드를 더 나눌 때 필요한 최소 데이터 개수다.

```
min_samples_split=24
```

이면:

데이터가 너무 적게 남은 노드는 무시해서 계속 쪼개지 말자.

라는 의미다.

역시 **과적합** 방지와 관련 있다.

17. Best Model

```
best_df_clf = grid_cv.best_estimator_
```

GridSearchCV가 찾은:

*가장 좋은 설정이 적용된 **Decision Tree** 모델*

을 가져온다.

참고로 변수명 `best_df_clf` 는 아마 `best_dt_clf` 라고 쓰려다 작성된 것으로 보여. 기능에는 문제없어.

```
pred1 = best_df_clf.predict(X_test)
```

최적 모델로 테스트 데이터를 예측한다.

결과:

```
0.86868
```

즉:

86.87%

기본 Decision Tree:

```
86.22%
```

에서 약간 올라갔다.

18. 가장 중요한 Feature 찾기

마지막:

```
ftr_importances_values = best_df_clf.feature_importances_
```

561개 Feature의 중요도를 가져온다.

```
pd.Series(  
    ftr_importances_values,  
    index=X_train.columns  
)
```

Feature 이름과 중요도를 연결한다.

```
.sort_values(ascending=False)[:20]
```

중요도가 높은 순서로 정렬해서:

만 가져온다.

그래프로 그리면:

사람의 행동을 구분하는 데 어떤 센서 Feature가 중요한지

볼 수 있다.

DAY2를 진짜 핵심 흐름으로 다시 압축하면

DAY1:

모델을 만들어 보자.

DAY2:

① Accuracy만 믿으면 안 된다.

② 그래서

Confusion Matrix

Precision

Recall

F1

ROC-AUC

를 사용한다.

③ predict()는 최종 답

predict_proba()는 확률이다.

④ Threshold를 바꾸면

Precision과 Recall이 달라진다.

⑤ 실제 당뇨병 데이터에 적용한다.

⑥ Decision Tree는

조건을 계속 나누며 분류한다.

- ⑦ Tree가 너무 복잡하면
Overfitting이 생길 수 있다.
- ⑧ max_depth,
min_samples_split,
min_samples_leaf
등으로 Tree를 조절한다.
- ⑨ GridSearchCV가
좋은 Hyperparameter를 자동으로 찾아준다.

그리고 지금 단계에서 **DAY2**에서 반드시 이해하고 넘어가야 할 건 딱 **6개**라고 봐.

1. **Confusion Matrix**의 TP/TN/FP/FN
2. **Precision**과 **Recall** 차이
3. `predict()` 와 `predict_proba()` 차이
4. **Threshold**를 낮추면 왜 **Recall**이 올라가는가
5. **Decision Tree**의 과적합과 `max_depth`
6. **GridSearchCV** = 좋은 **Hyperparameter** 조합 찾기

나머지 `visualize_boundary()` 내부나 중복 컬럼 이름을 처리하는 복잡한 함수는 **현재 수준에서 한 줄 한 줄 암기할 필요 없어**. 수업 흐름상 핵심 개념을 보여주기 위한 보조 코드에 더 가깝다.

2026. 8. 14. 오후 2:34:08